

Генератор интеграций с платёжными провайдерами

Инструмент, который читает документацию платёжного провайдера и собирает из неё готовую интеграцию для Space Payments: сервис, инструкцию, тестовые примеры и список того, что осталось проверить человеку.

Сегодня показываем, как инструмент читает документацию.
Генерацию кода привезём на второй чекпоинт. Хотим сверить с вами направление.

Новый провайдер сейчас стоит 2–5 дней ручной работы

Что есть на входе

Файл с описанием API провайдера: куда слать запросы, какие поля передавать, какие ответы ждать.

Что нужно на выходе

Готовый сервис по вашему контракту, инструкция по подключению, тестовые примеры запросов и ответов. Одной командой, без ручных шагов.

В задании один провайдер, NovaPay. В жизни их будет много.

Инструмент должен справиться со следующим провайдером, документацию которого мы ещё не видели. Поэтому мы делаем не интеграцию с NovaPay, а генератор. NovaPay для него первый пример.

Вопрос к вам



Мы правильно понимаем задачу? И на защите вы проверите решение на новой спецификации или только на NovaPay?

2. Что взяли за основу

Инструмент работает как переводчик

Сначала понимает документацию, потом пишет код. Между ними нейтральное описание провайдера.

1

Читает документацию

Находит запросы, поля, авторизацию, статусы, ошибки, уведомления. Про каждый факт помнит, откуда он взят.

2

Переводит на нейтральный язык

Не «поле amount у NovaPay», а «сумма выплаты». Внутри инструмента нет ничего от конкретного провайдера.

3

Собирает файлы по шаблонам

Сервис на Ruby, инструкция, тестовые примеры и отчёт о том, что осталось уточнить.

Знания о провайдерах лежат в справочниках, а не в коде: названия статусов, валюты, заголовки подписи. Новый провайдер добавляется строками в таблицу, код не трогаем.

Вопрос к вам



Настоящего класса `Provider::BaseService` у нас нет, мы восстановили его по примеру из задания. Расскажите, как устроены `approve_operation`, `reject_operation` и `client`?

3. Какую лазейку нашли

Нейросети запрещены, а документацию понимать надо

Выручили стандарты: почти всё, что в документации написано словами, давно описано открытым стандартом.

Сумма в копейках или в рублях?

ISO 4217. У каждой валюты стандартное число знаков после запятой. У рубля два, значит множитель 100. Слово «копейках» используем только для проверки.

БИК обязателен только для СБП. Как учесть?

JSON Schema. Такое условие можно записать формально. Если оно есть только словами, инструмент замечает и просит человека подтвердить.

Как проверять подпись уведомления?

Standard Webhooks. Отраслевая спецификация подписи. Схема NovaPay от неё отличается, поэтому проверка настраиваемая.

Что значит ответ 409 при повторе?

Idempotency-Key, черновик IETF. Повтор возвращает уже созданную выплату. Это не ошибка, а защита от дублей.

Куда записать то, чего в документации нет?

OpenAPI Overlay. Официальный формат дополнений к спецификации, а не наш самодельный файл настроек.

Как понять, что поле `sum` это сумма?

Сопоставление схем баз данных. Словари синонимов, сходство названий, тип поля, каждое с весом. Методы 2001 года.

Один файл на входе всегда даёт один и тот же результат. Это проверяют автотесты, и это же доказывает, что нейросети внутри нет.

3. Что сделали там, где стандарт не помогает

Главный принцип: инструмент не угадывает молча

Про каждый факт он знает, откуда его взял, и ведёт себя по-разному.

- | | | |
|----------|---|---|
| 1 | Написано в файле явно. Обязательные поля, допустимые значения, типы. | Берём как есть. |
| 2 | Взято из стандарта или справочника. Множитель валюты, перевод статуса. | Берём, но записываем источник и уверенность. |
| 3 | Понять не удалось. Условие описано словами, статус незнакомый. | Не придумываем. Пишем в отчёт, оставляем пометку в коде и шаблон для ответа человека. |

Незамеченная ошибка в платёжной интеграции дороже минуты ручной проверки.

Вопрос к вам



Такой баланс вам подходит: очевидное автоматически, спорное отдаём человеку с подсказкой? Или вы ждёте, что инструмент будет решать больше сам, пусть и с риском ошибиться?

Шесть мест, где простой генератор ошибся бы молча

- 1** **Ответ 409 значит две разные вещи.** При создании выплаты: «такая уже есть, вот она». При отмене: «отменить нельзя». Простой генератор отклонил бы уже созданную выплату.
- 2** **Три кода ошибок есть в примерах, но нет в официальном списке:** `unauthorized`, `not_found`, `invalid_status`. Учитываем и список, и примеры.
- 3** **Три кода из списка нет ни в одном примере.** Обработку для них всё равно делаем, расхождение пишем в отчёт.
- 4** **У запроса статуса не описаны ответы 429 и 500,** хотя у создания выплаты они есть. Добавляем стандартную обработку.
- 5** **«БИК обязателен для СБП» написано только словами.** Инструмент это находит, но не выдаёт за факт: просит подтвердить.
- 6** **Подпись уведомления описана не полностью.** Заголовок и алгоритм есть, формат подписи и что подписывается, нет. Взяли вариант, который вы подтвердили.

Вопрос к вам



Про 409 при повторе: сервис должен взять уже созданную выплату и продолжить с ней, а не отклонять операцию. Подтверждаете?

5. Вот смотрите, как оно работает

Одна команда, один экран

```
$ ./integrate analyze --spec provider_api.yaml
Разбор спецификации... novapay.yaml: OpenAPI 3.0.3, 5 операций, 8 схем, 31 поле
Провайдер: novapay (эвристика 0.80)
Авторизация: ApiKeyAuth -> api_key, заголовок X-API-Key, ключи credentials: api_key
Единицы суммы: minor, x100 (ISO 4217: экспонента RUB 2); валюта RUB (задано явно 1.00)
Статусы: 5 сопоставлено, 0 не выведено
  pending    -> in_progress (по справочнику 1.00)
  processing -> in_progress (по справочнику 1.00)
  completed  -> approved (по справочнику 1.00)
  failed     -> rejected (по справочнику 1.00)
  cancelled  -> rejected (по справочнику 1.00)
Вебхук: /webhooks/payout - 4 события, подпись X-NovaPay-Signature (custom: hmac_sha256, hex, raw_body)
  payout.completed -> approved (по справочнику 1.00)
  payout.failed    -> rejected (по справочнику 1.00)
  payout.processing -> in_progress (по справочнику 1.00)
  payout.cancelled -> rejected (по справочнику 1.00)
Идемпотентность: заголовок Idempotency-Key (необязательный), стратегия uuid_v5, dedup on 409
Ошибки: 7 в enum + 3 только в примерах; дедупликация 409 у createPayout
  unauthorized      alert (по справочнику 0.80) [пример]
  not_found         reject (по справочнику 0.80) [пример]
  invalid_status    reject (по справочнику 0.80) [пример]
  createPayout      400 reject, 401 alert, 402 escalate, 409 dedup, 422 reject, 429 retry_backoff +Retry-After, 500 retry_backoff
Условия взаимодействия: 9
  retry_after      createPayout 429 (задано явно 1.00)
  min_amount       amount      100000 (задано явно 1.00)
  field_max_length external_id 64 (задано явно 1.00)
  cancel_status_restriction cancelPayout pending, processing (эвристика 0.60)
Предупреждения: 14 (0 ошибок, 5 предупреждений, 9 справок)
```

Первый критерий, разбор спецификации, закрыт на живом прогоне

5

запросов к API и
назначение каждого

два из них вне вашего
контракта, об этом сказано явно

31

поле в 8 структурах
данных

тип, обязательность,
ограничения

1

способ авторизации
ключ в заголовке X-API-Key

×100

множитель суммы:
копейки
по ISO 4217, у рубля два знака

5 из 5

статусов переведены в
ваши
по таблице из задания

7 + 3

кодов ошибок
7 в официальном списке, ещё 3
только в примерах

4

вида уведомлений и их
подпись
заголовок X-NovaPay-Signature,
HMAC-SHA256

9

условий взаимодействия
пауза Retry-After, минимальная
сумма, отмена только в двух
статусах, защита от дублей по
409

Каждое сомнение инструмента: где, в чём и как исправить

Строка отчёта указывает на место в документации, объясняет проблему и предлагает готовый фрагмент. Человек вписывает одно значение, инструмент перечитывает файл, предупреждение исчезает.

- **Ошибка.** Без ответа человека сервис собирать нельзя.
- **Предупреждение.** Сервис собран, но это место стоит проверить глазами.
- **Справка.** Осознанное решение, например запрос, которого нет в контракте.

```
ВНИМАНИЕ $.components.schemas.Recipient
`bank_code` выглядит условно обязательным (`type` = sbp),
но схема не задаёт условия; фрагмент ниже задаёт его формально
overlay:
  - target: "$.components.schemas.Recipient"
    update:
      x-jsonschema-if:
        properties:
          type:
            const: sbp
      x-jsonschema-then:
        required: [bank_code]
```

Разбор документации работает, генерации кода пока нет

Готово к первому чекпоинту

- Чтение спецификаций в YAML и JSON. На битый файл инструмент отвечает понятным сообщением, а не падает.
- Полный разбор: запросы, поля, авторизация, суммы, статусы, ошибки, уведомления, защита от дублей.
- Экран разбора с объяснением каждого решения, на русском и английском.
- Автотесты проходят.

Ко второму чекпоинту

- Сопоставление полей: какое поле провайдера что означает.
- Генерация всех четырёх файлов.
- **Тестовый сервер провайдера из той же документации:** сгенерированный сервис отправит настоящий запрос и получит настоящее уведомление.
- Ещё три спецификации: карта вместо СБП, депозиты в рублях вместо копеек, намеренно дырявая документация.

Вопрос к вам



Что вам важнее увидеть на второй встрече: глубокую работу на NovaPay или работу на трёх разных провайдерах?

Пять вопросов. От ответов зависит, что мы напишем дальше.

- 1 Настоящего `Provider::BaseService` у нас нет, мы восстановили его по примеру из задания. Сколько аргументов принимают `approve_operation` и `reject_operation`, и что такое `client`: это ваш HTTP-клиент? От этого зависит вся генерация сервиса. Если узнаем сейчас, поправим за минуты.
- 2 Ответ 409 при повторном создании выплаты: сервис берёт уже созданную выплату и продолжает с ней, а не отклоняет операцию? Разница между рабочей интеграцией и потерянными деньгами клиента.
- 3 Отменённая выплата у вас считается отклонённой. Так и останется, или появится отдельный статус «отменено»? От ответа зависит набор внутренних статусов.
- 4 На защите будет новая спецификация или только NovaPay? Меняет приоритет: устойчивость к незнакомому или глубина на известном.
- 5 Достаточно ли примеров в `fixtures.json`, или нужны исполняемые тесты? Считается ли сгенерированный код в доле Ruby? Нужен ли Docker?

Как соблюдаем правила кейса

Нейросетей внутри нет

Только справочники, правила и сравнение строк. Один вход всегда даёт один выход, это проверяют автотесты. С нейросетью внутри такое доказать было бы невозможно.

Всё на Ruby

Ядро целиком на Ruby. Веб-интерфейс, если понадобится, будет тонким и без отдельной сборки на JavaScript.

Только открытые библиотеки

Каждая зависимость записана вместе с лицензией и назначением. Сам инструмент не ходит ни в какие внешние сервисы.

Никаких секретов в коде

Сгенерированный сервис читает ключи только из настроек платформы. В тестовых примерах заглушки и тестовые номера карт.